

Lab 13 — Hash Tables, Twice

DSA27 Lab Manual · Week 13 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Fall 2026 · Week 13

Contents

Part 0 — Before you start	2
Part 1 — Hashing by hand	4
Part 2 — Chaining: get and put	5
Part 3 — Chaining: delete	6
Part 4 — Chaining: <code>_resize</code>	7
Part 5 — Open addressing: <code>_probe</code> , put, get	9
Part 6 — Tombstones, and breaking them on purpose	10
Part 7 — Open addressing: <code>_resize</code>	12
Part 8 — Measure it	13
Part 9 — Check against dict, the oracle	15
Part 10 — Exercises at a glance	15
Part 11 — Take-home practice (not graded)	16
Part 12 — Bridge to Lecture 14: graphs	17
Summary	17
Answers to the checkpoints	18

How to use this lab. The routine from Week 4: read the lecture section named at the top of each part, **draw before you code** — here that means a row of 8 slots on paper, with the keys in them and the probes as arrows — and predict at each **Checkpoint** (answers at the end). Then write one method, run the tests named in its part, and only then move on. You build the same map twice: first with chains, then with probing and tombstones. Parts 1–7 are the session; Parts 8–10 are for home.

Duration	One 2-hour lab session, plus about 3 hours at home
You will write	<code>dsa/hashtable.py</code> — <code>ChainingHashMap.get</code> , <code>put</code> , <code>delete</code> , <code>_resize</code> ; <code>OpenAddressingHashMap._probe</code> , <code>put</code> , <code>get</code> , <code>delete</code> , <code>_resize</code>
Graded by	<code>tests/test_hashtable.py</code> (17 tests)
Connects to	Lecture 13 — Hash tables; Week 5 (the Entry chain is a linked list); Week 4 (resizing is amortised); Week 14, whose Graph is built on your <code>ChainingHashMap</code>

What you will be able to do

1. compute `hash(key) % capacity` by hand for integer keys, and say which hashes are the same in every run and which are not;
 2. draw a chaining table and an open-addressing table after every operation;
 3. write `get`, `put` and `delete` for a chaining table, with the head of a chain as the special case;
 4. write `_resize` so that every key is **rehashed**, and show what goes wrong when buckets are copied across instead;
 5. write linear probing as a generator, and `put/get/delete` that stop at `None` but not at a tombstone;
 6. explain, with the failing test in front of you, why an open-addressing delete must leave a tombstone;
 7. count probes against the load factor, and make a hash table slow on purpose with keys that all collide.
-

Part 0 — Before you start

0.1 Environment and prerequisites

From the DSA27 folder, with the virtual environment active (the prompt starts with `(.venv)`):

```
git pull
pytest -m "not challenge" -q      # environment check: must pass
pytest tests/test_hashtable.py -q # 17 failures: nothing written
```

The 17 failures are all `NotImplementedError` — that is the starting point.

What you need from earlier weeks:

- **The course Array** (Lecture 02) — both tables store everything in it.

- **Linked-list surgery** (Week 5): walking a chain with `entry = entry.next`, pushing on the front, and unlinking with a `prev` reference.
- **Generators** (Lab 03): `_probe` is one — a function with `yield`.
- **The % wrap** of the ring buffer (Week 7).

Nothing from `dsa/` is imported except `Array`: unlike the queues, this week stands on no exercise of yours.

0.2 Read the skeleton: what is given, what is yours

Open `dsa/hashmap.py` and read it from the top. Given to you:

Name	What it is
<code>_MISSING</code>	a private object meaning “no default was passed to <code>get</code> ”
<code>TOMBSTONE</code>	the marker an open-addressing delete leaves behind
<code>Entry</code>	a chain node: <code>key</code> , <code>value</code> , <code>next</code>
<code>both __init__</code>	the <code>Arrays</code> , <code>_size</code> , <code>max_load</code> (and <code>_tombstones</code> for open addressing)
<code>ChainingHashMap._index</code>	<code>hash(key) % len(self._buckets)</code>
<code>load_factor</code> , <code>__len__</code> , <code>__contains__</code> , <code>__iter__</code>	on both classes

Yours: the four methods of `ChainingHashMap` and the five of `OpenAddressingHashMap` listed at the top of this lab. Read the given `ChainingHashMap.__iter__` now: it walks every chain of every bucket, which is the loop your `_resize` will need.

0.3 The storage rule, this week

A hash table built on a Python `dict` would be a joke with a straight face. The storage rule says a `dsa/` structure keeps its data only in the course `Array`, in node objects, or in another `dsa/` structure. So:

- `ChainingHashMap`: the buckets are an `Array`; the chains are `Entry` nodes.
- `OpenAddressingHashMap`: two `Arrays`, `_keys` and `_values`, side by side: slot `i` of one belongs with slot `i` of the other.
- No `list`, `dict` or `set` inside `dsa/hashmap.py` — not even “just for `_resize`”. A `resize` builds new `Arrays` and walks the old ones.

0.4 The tests, and a -k note

Eight tests are **parametrised** over both classes, so each runs twice; the class name is in the test id. One more test is only for open addressing:

8 tests, then 9 tests:

```
pytest tests/test_hashmap.py -v -k Chaining
```

```
pytest tests/test_hashmap.py -v -k "OpenAddressing or open_addressing"
```

-k matches case-insensitively, but not across the underscore: -k OpenAddressing selects 8 tests, and misses test_open_addressing_delete_preserves_probe_chain — the one that matters most. Use the second filter above.

Part 1 — Hashing by hand

Lecture 13, “Hash Functions”. Before writing a table, get a feel for the one line that is given to you: `hash(key) % capacity`.

1.1 At the REPL

```
>>> hash(42), hash(-1), hash(3.0), hash(True)
>>> hash((1, 2)) == hash((1, 2))
>>> hash("dsa")
>>> hash([1, 2])
```

Now quit the REPL and run this line **twice**, from PowerShell:

```
python -c "print(hash('dsa'), hash(12345), hash((1, 2)))"
python -c "print(hash('dsa'), hash(12345), hash((1, 2)))"
```

The string’s hash changes from run to run; the other two do not (Lecture 13, “Same run, same hash”). Then fix the seed and run it twice more:

```
$env:PYTHONHASHSEED = "0"
python -c "print(hash('dsa'))"
python -c "print(hash('dsa'))"
Remove-Item Env:PYTHONHASHSEED
```

With a fixed seed the string’s hash is the same both times. Remember this command: in Part 6 you will need it.

Checkpoint 1. Without running anything:

- (a) For a table of capacity 8, give the index of the keys 7, -3, 24 and $2^{61} - 1$. (Python’s % is never negative here.)
- (b) `m = ChainingHashMap()`, then `m.put(3, "a")`, `m.put(3.0, "b")`, `m.put(True, "t")`, `m.put(1, "one")`. What are `len(m)`, `m.get(3)` and `m.get(True)`? Why?
- (c) Why does every trace in this lab use integer keys?

Part 2 — Chaining: get and put

Lecture 13, “A linked list in every bucket” and “The three operations, in words”.

2.1 The idea

Each slot of `self._buckets` is `None` or the first `Entry` of a chain. `get` walks the chain at `self._index(key)`; `put` walks the same chain to see whether the key is already there, and if it is not, pushes a new `Entry` on the front.

2.2 Draw it

Draw 8 boxes in a row, numbered 0 to 7. Put the keys in, one at a time, each at the **front** of its chain (the value does not matter; write only the key):

Checkpoint 2. A `ChainingHashMap()` (*capacity 8, max_load 0.75*) receives `put` for the keys 3, 11, 6, 19, 14, in that order.

(a) Draw the buckets. Which chains have more than one entry, and in what order?

(b) What is the load factor? Has the table resized?

(c) `get(27)`: which bucket, how many entries are compared, and what happens?

2.3 Write get

1. Find the chain: `entry = self._buckets[self._index(key)]`.
2. Walk it: while `entry` is not `None`, if `entry.key == key`, return its value; otherwise move to `entry.next`.
3. At the end of the chain: if `default` is `_MISSING`, **raise `KeyError(key)`**; otherwise return `default`.

Compare keys with `==`, never `is`: two equal strings can be different objects. And compare `default` with `is _MISSING`: `None` is a default a caller is allowed to pass.

2.4 Write put — without resizing, first

1. Find the chain and walk it. If you meet the key, **overwrite** its value and return — the size does not change.
2. Not found: `self._buckets[i] = Entry(key, value, self._buckets[i])` — one line that pushes on the front (Week 5’s `push_front`).
3. `self._size += 1`.
4. If `self._size / len(self._buckets) > self.max_load`, call `self._resize(2 * len(self._buckets))`.

Step 4 calls a method you have not written yet. That is deliberate: until the table passes 6 keys, it never runs.

2.5 Test it

```
pytest tests/test_hashmap.py -v -k Chaining
```

With `get` and `put` written, you should see **4 passed** and 4 failed: `put_then_get`, `put_overwrites_without_growing`, `get_missing_key` and `collisions_are_handled` pass. `test_delete` and `test_delete_missing_key_raises` fail with `NotImplementedError` from `delete` (Part 3), and `test_survives_many_keys_and_resizes` and `test_load_factor_stays_bounded` fail with `NotImplementedError` from `_resize` (Part 4).

2.6 When it fails

Bug	What you see	Fix
no “already there?” walk in <code>put</code>	<code>assert 2 == 1</code> in <code>test_put_overwrites_without_growing</code> — <code>len</code> counted a duplicate	walk the chain first; <code>overwriting</code> and return
return <code>None</code> instead of raise <code>KeyError</code> in <code>get</code>	Failed: DID NOT RAISE <code>KeyError</code> in <code>test_get_missing_key</code> (and later in <code>test_delete</code>)	only default is <code>_MISSING</code> raises; anything else is returned
if default: or if default is <code>None</code> :	<code>get(key, None)</code> or <code>get(key, 0)</code> raises instead of returning	the sentinel is <code>_MISSING</code> ; test with <code>is</code>
<code>self._buckets[i] = Entry(key, value)</code> (no third argument)	keys that collide disappear: <code>KeyError</code> in <code>test_collisions_are_handled</code>	the new entry’s next is the old head

Part 3 — Chaining: delete

3.1 The idea

Week 5’s `remove`, on a chain whose head lives in an `Array` slot. Walk with two references, `prev` (starting at `None`) and `entry`. When `entry.key == key`:

- if `prev` is `None`, the match is the **head**: the bucket slot itself becomes `entry.next`;
- otherwise `prev.next = entry.next` skips it.

Then `self._size -= 1` and return. If the walk ends: `raise KeyError(key)`.

Checkpoint 3. Take the table of Checkpoint 2. Draw it after `delete(11)`, then after `delete(19)`. Which of the two is the head case? What would `delete(27)` do?

3.2 Write it, then test it

```
pytest tests/test_hashmap.py -v -k "Chaining and delete"
```

Two tests: `test_delete` and `test_delete_missing_key_raises`.

3.3 When it fails — and when it does not

Bug	What you see	Fix
<code>no self._size -= 1</code>	<code>assert 1 == 0</code> in <code>test_delete</code>	the size counts keys, and one just left
<code>self._buckets[i] = entry.next</code> for every match, head or not	nothing — both tests pass	see below

The second bug is the lesson of this part. Unlinking a match in the middle of a chain by writing `self._buckets[i] = entry.next` throws away every entry **before** it, silently. The tests delete from a table of one key, where the match is always the head, so they cannot see it. Write your own check, at the REPL or in the notebook:

```
from dsa.hashmap import ChainingHashMap
m = ChainingHashMap()
for k in [3, 11, 19]:          # one chain: 19 -> 11 -> 3
    m.put(k, k)
m.delete(11)                  # the middle of the chain
print(len(m), m.get(19), m.get(3))  # must print: 2 19 3
```

A test suite is a set of examples. When the examples cannot reach a line of your code, you write the example that does.

Part 4 — Chaining: `_resize`

Lecture 13, “Resize: every key moves”.

4.1 The idea

`_resize(capacity)` swaps in a new, empty `Array(capacity)` and moves every entry of the old one into it. Each entry must go to its **new** index, `hash(key) % capacity`, which is usually not its old one. Re-link the existing `Entry` nodes; there is no need to create new ones.

Checkpoint 4. A `ChainingHashMap(capacity=4)` receives `put` for 1, 5, 2, 6.

(a) Draw the buckets after each `put`. After which `put` does the table resize, and to what

capacity?

(b) Draw the table after the resize.

(c) Suppose `_resize` had copied `old[i]` into `new[i]` instead. What would `get(5)` do, and why is `len(m)` still right?

4.2 Write it

1. Keep the old array: `old = self._buckets`; then `self._buckets = Array(capacity)`. From here on, `self._index` uses the new capacity — which is exactly what you want.
2. For each chain in `old`, walk it. For each entry:
 - save `nxt = entry.next` **first**;
 - compute `i = self._index(entry.key)`;
 - push the entry on the front of the new chain `i`: `entry.next = self._buckets[i]`, then `self._buckets[i] = entry`;
 - move on to `nxt`.
3. `_size` does not change: the same keys, in more buckets.

4.3 Test it

```
pytest tests/test_hashmap.py -v -k Chaining
```

All 8 must pass. `test_survives_many_keys_and_resizes` puts 500 **string** keys ("key0" to "key499"), so the table resizes seven times, from 8 to 1,024 buckets; `test_load_factor_stays_bounded` puts 500 integers and checks the load factor is at most `max_load + 0.05` at the end.

4.4 When it fails

Bug	What you see	Fix
buckets copied across, <code>new[i] = old[i]</code>	<code>KeyError: 'key0' (or another key) in</code> <code>test_survives_many_keys_and_resizes</code>	recompute every index with the new capacity
<code>entry = entry.next</code> after relinking the entry	<code>KeyError</code> in the same test: the relink overwrote <code>next</code> , so the walk jumps into the new table and the rest of the old chain is lost	save <code>nxt</code> before you relink
never calling <code>_resize</code> in <code>put</code>	<code>assert 62.5 <= (0.75 + 0.05)</code> in <code>test_load_factor_stays_bounded</code> — 500 keys in 8 buckets	the check in step 4 of Part 2.4

Bug	What you see	Fix
<code>self._index</code> computed before replacing <code>self._buckets</code>	the same <code>KeyError</code> as the copy	swap the array first, then compute indices

The exact key in the `KeyError` can change from run to run: the keys are strings, and their hashes are randomised (Part 1). The failure itself does not.

Part 5 — Open addressing: `_probe`, `put`, `get`

Lecture 13, “Linear probing: one key per slot”.

5.1 The idea

No chains: `_keys[i]` holds a key, `None` (never used) or `TOMBSTONE` (a deleted key was here); `_values[i]` holds the value that goes with `_keys[i]`. A key tries slots in the order its **probe sequence** gives:

$$h, h + 1, h + 2, \dots \pmod{\text{capacity}}, \quad h = \text{hash}(\text{key}) \pmod{\text{capacity}}$$

5.2 Draw it

Checkpoint 5. An `OpenAddressingHashMap()` (capacity 8, max_load 0.66) receives `put` for 3, 11, 19, 4.

- Draw the 8 key slots after each `put`. How many slots does each `put` probe?
- `get(12)`: list the slots probed. Where does it stop, and what does it return?
- Key 4 hashes to 4, yet sits somewhere else. What does that tell you about how `get(4)` must search?

5.3 Write `_probe`

A generator: compute `start = hash(key) % capacity` once, then for `step` in `range(capacity)`: `yield (start + step) % capacity`. Exactly capacity indices, every slot once, wrapping round the end.

5.4 Write `get` (and a helper, if you like)

Loop for `i` in `self._probe(key)`:

- `self._keys[i]` is `None` → a never-used slot: the key is **not** in the table. Stop.
- `self._keys[i]` is `TOMBSTONE` → **skip it**: keep probing.
- `self._keys[i] == key` → found: return `self._values[i]`.

If the loop ends, or stops at `None`: default, or `KeyError`, exactly as in the chaining `get`. `delete` will need the same search, so it is sensible to write it once as a helper — say `_find(key)`, returning the slot index or `-1` — and call it from both.

5.5 Write put

1. **Load check first.** If $(\text{self}._\text{size} + \text{self}._\text{tombstones} + 1) / \text{capacity}$ would exceed `max_load`, resize before inserting: to **double** the capacity if $(\text{self}._\text{size} + 1) / \text{capacity}$ alone exceeds it, otherwise to the **same** capacity (which just sweeps away the tombstones). Tombstones must be counted: Lecture 13, “Delete: why not just empty the slot?”, last paragraph.
2. Probe. Remember the **first** tombstone you pass. If you meet the key, overwrite its value and return. If you meet `None`, stop.
3. Insert at the first tombstone you passed if there was one, otherwise at the `None` where you stopped. If you reused a tombstone, `_tombstones -= 1`. Then `_size += 1`.

Until Part 7, `_resize` raises `NotImplementedError`; with the load check first, the fifth distinct key is fine and the sixth calls it.

5.6 Test it

```
pytest tests/test_hashmap.py -v -k "OpenAddressing or open_addressing"
```

With `_probe`, `get` and `put` written, `put_then_get`, `put_overwrites_without_growing`, `get_missing_key` and `collisions_are_handled` pass. The tests that delete or resize wait for Parts 6 and 7.

Part 6 — Tombstones, and breaking them on purpose

Lecture 13, “Delete: why not just empty the slot?”. This is the part of the week that the TA will ask you to explain.

6.1 Write delete

Find the key’s slot with the same search as `get`. Not there: `KeyError`. There: `set_keys[i] = TOMBSTONE` and `_values[i] = None` (so the old value can be freed), `_size -= 1`, `_tombstones += 1`.

Checkpoint 6. Continue the table of Checkpoint 5 (3, 11, 19, 4 in slots 3, 4, 5, 6).

(a) `delete(11)`. Draw the slots. Then `get(19)`: list the slots probed.

(b) Now `put(12)`. Does the load check trigger a resize? List the probes and say where 12 goes.

(c) Go back to (a), but suppose `delete` had written `None` instead of `TOMBSTONE`. What

do get(19) and get(4) do now?

6.2 Test it

```
pytest tests/test_hashmap.py -v -k "OpenAddressing or open_addressing"
```

test_delete, test_delete_missing_key_raises and test_open_addressing_delete_preserve should now pass too.

6.3 Break it

Change one word in your delete: `self._keys[i] = None` instead of `self._keys[i] = TOMBSTONE`.

Checkpoint 7. Before you run the tests: of the 17, which fail? Write down their names. Why do the other open-addressing tests that delete keys still pass?

Run all 17 and compare:

```
pytest tests/test_hashmap.py -v
```

Read the failing test's docstring and its four lines. Then **put TOMBSTONE back**, and run the tests again. If you are not sure you restored exactly what you had, `git diff dsa/hashmap.py` shows every line you changed.

6.4 When it fails

Bug	What you see	Fix
delete writes None	KeyError: 8 in test_open_addressing_delete_preserves_probe_chain	TOMBSTONE
get stops at a tombstone as if it were None	the same KeyError: 8	a tombstone means "keep going"
_probe yields start + step with no % capacity	IndexError: Array index 8 out of range for length 8 — or 16, 64, 128, 1024, in a different test on a different run	wrap: (start + step) % capacity
put inserts at the first tombstone without probing on	no test fails; the table can hold the same key twice	keep probing to None before inserting

The third row deserves a minute. Which test fails, and with which length, depends on **where the string keys land** — and string hashes change every run. Run the tests three times and you may see three different failures, and even a run where test_delete fails for the key "a". When a failure moves around like that, fix the seed to reproduce it:

```
$env:PYTHONHASHSEED = "0"
pytest tests/test_hashmap.py -v -k OpenAddressing
Remove-Item Env:PYTHONHASHSEED
```

For the fourth row, write your own test, as in Part 3.3:

```
from dsa.hashmap import OpenAddressingHashMap
m = OpenAddressingHashMap()
m.put(0, "a"); m.put(8, "b")      # both hash to 0: slots 0 and 1
m.delete(0)                      # slot 0 is now a tombstone
m.put(8, "B")                   # 8 is already in slot 1: overwrite it
print(len(m), m.get(8))         # must print: 1 B
```

Part 7 — Open addressing: `_resize`

7.1 Write it

1. Keep the old `_keys` and `_values`; make two new `Array(capacity)s`.
2. Reset `_size` **and** `_tombstones` to 0 — you are about to rebuild both counts.
3. For every old slot whose key is neither `None` nor `TOMBSTONE`, insert it: probe the new table and put it in the first `None`, then `_size += 1`. The new table has no tombstones and no duplicates, so the first `None` is always right. (Calling `self.put` would also work, but it re-runs the load check and the duplicate search for nothing.)

7.2 Test it

```
pytest tests/test_hashmap.py -v
```

All 17 must pass.

7.3 When it fails

Bug	What you see	Fix
<code>_size</code> not reset before re-inserting	<code>assert 1175 == 500</code> (the number varies) in <code>test_survives_many_keys_and_resizes</code>	reset, then count up again
the load check always resizes to the same capacity	<code>TypeError: 'NoneType' object cannot be interpreted as an integer</code> in two tests — the table filled up, no <code>None</code> was left, and the insert index was never set	double when the live keys alone exceed <code>max_load</code>

Bug	What you see	Fix
tombstones copied into the new table	no test fails, but the table never gets rid of them	skip TOMBSTONE as well as None

Checkpoint 8. In a fresh `OpenAddressingHashMap()` (capacity 8), run `put(k, k)` then `delete(k)` for $k = 0, 1, 2, \dots, 7$.

- (a) With the reference load check, how many tombstones are in the table after the last delete, and in which slots? When were the others swept away?
- (b) Now imagine a load check that counts only **live** keys. What does the table look like after the last delete? How many slots does `get(100)` probe? What would a `while self._keys[i] is not None: loop` do?

Part 8 — Measure it

Lecture 13, “Measured”. Two experiments, in `notebooks/13-hash-tables.ipynb`.

8.1 Probes against load factor

Count probes by **subclassing**: the subclass’s `_probe` passes on every index that yours yields, and counts them. Your code does not change.

```
import random
from dsa.hashmap import OpenAddressingHashMap

class Counting(OpenAddressingHashMap):
    probes = 0
    def _probe(self, key):
        for i in super()._probe(key):
            Counting.probes += 1
        yield i

capacity = 4096
rng = random.Random(13)
keys = rng.sample(range(10**9), capacity)
misses = rng.sample(range(10**9, 2 * 10**9), 1000) # never in the table
for alpha in [0.25, 0.5, 0.7, 0.8, 0.9]:
    m = Counting(capacity=capacity, max_load=0.99) # no resizing
    n = int(alpha * capacity)
    for k in keys[:n]:
        m.put(k, k)
    Counting.probes = 0
    for k in keys[:n]:
```

```

        m.get(k)
    hit = Counting.probes / n
    Counting.probes = 0
    for k in misses:
        m.get(k, None)
    miss = Counting.probes / 1000
    print(f"alpha {alpha:4}: hit {hit:5.2f} miss {miss:6.2f}")

```

With the reference solution it prints hit/miss averages of 1.16/1.36, 1.53/2.61, 2.22/6.22, 3.16/12.78 and 5.00/44.83. Yours should match exactly if your `_probe` is linear probing and your `get` counts the same way: the keys are fixed by the seed, and integer hashes do not change between runs. Compare with the lecture's table and with Knuth's formulas, $\frac{1}{2}(1 + \frac{1}{1-\alpha})$ for a hit and $\frac{1}{2}(1 + \frac{1}{(1-\alpha)^2})$ for a miss. With 4,096 slots your numbers will scatter more than the lecture's 16,384 — at 0.9 especially — but the shape must be the same: flat up to about 0.7, then a cliff.

8.2 Make it slow on purpose

```

import time
from dsa.hashmap import ChainingHashMap

def time_gets(keys):
    m = ChainingHashMap()
    for k in keys:
        m.put(k, k)
    start = time.perf_counter()
    for k in keys[:200]:
        m.get(k)
    return (time.perf_counter() - start) / 200 * 1e6 # us per get

for n in [500, 1000, 2000, 4000]:
    good = random.Random(n).sample(range(10**9), n)
    bad = [i << 20 for i in range(n)] # i * 2**20
    print(f"n {n:5}: random {time_gets(good):7.2f} us"
          f"    bad {time_gets(bad):8.2f} us")

```

With random keys, a `get` costs about the same at every `n`. With the bad keys — every one a multiple of 2^{20} , so `key % capacity` is 0 for every capacity the table will ever have — each `get` walks one long chain, and the time doubles when `n` doubles. On the lecture's machine: about 1.3 μ s against 190 μ s at `n = 4,000`. Building the bad table is itself $O(n^2)$, so do not go much past 4,000.

Then try the same bad keys in a Python dict, and explain why it does not slow down (Lecture 13, "Python's dict and set").

Part 9 — Check against dict, the oracle

Only now, with all 17 tests passing, bring in Python's dict — as a **second opinion** in a test script of your own, never inside dsa/. Random operations, the same on both, with small integer keys so that collisions, deletes and re-inserts happen constantly:

```
import random
from dsa.hashmap import ChainingHashMap, OpenAddressingHashMap

for cls in (ChainingHashMap, OpenAddressingHashMap):
    rng = random.Random(1)
    mine, oracle = cls(), {}
    for step in range(20_000):
        key = rng.randrange(64)
        op = rng.random()
        if op < 0.5:
            mine.put(key, step)
            oracle[key] = step
        elif op < 0.8 and key in oracle:
            mine.delete(key)
            del oracle[key]
        else:
            assert mine.get(key, None) == oracle.get(key), (cls.__name__, step, key)
            assert len(mine) == len(oracle), (cls.__name__, step)
    assert sorted(mine) == sorted(oracle)
    print(cls.__name__, "agrees with dict on 20,000 random operations")
```

This exercises exactly what the graded tests do not: deleting from the middle of a chain (Part 3.3), re-inserting over tombstones (Part 6.4), and tables that fill with tombstones and are swept (Checkpoint 8). If an assertion fails, the message gives the step and the key; replay the operations up to that step and draw the table.

Part 10 — Exercises at a glance

Every method is $O(1)$ on average except the two `_resizes`, which are $O(n)$ and make put amortised $O(1)$.

Class	Method	The trap	-k filter
Chaining	get	<code>==</code> not <code>is</code> ; <code>KeyError</code> only when default is <code>_MISSING</code>	Chaining (8)
Chaining	put	overwrite, never duplicate; resize when the load passes <code>max_load</code>	Chaining
Chaining	delete	the head of a chain; <code>size -= 1</code> ; the middle of a chain (untested)	Chaining and delete (2)

Class	Method	The trap	-k filter
Chaining	<code>_resize</code>	rehash every key; save next before relinking	Chaining
Open addr.	<code>_probe</code>	% capacity; exactly capacity indices	see below (9)
Open addr.	<code>get</code>	stop at None, skip TOMBSTONE	see below
Open addr.	<code>put</code>	count tombstones in the load check; probe on past a tombstone before inserting	see below
Open addr.	<code>delete</code>	leave TOMBSTONE; update both counts	see below
Open addr.	<code>_resize</code>	reset <code>_size</code> and <code>_tombstones</code> ; skip None and TOMBSTONE	see below

For open addressing the filter is `-k "OpenAddressing or open_addressing"`.

All 17 at once:

```
pytest tests/test_hashmap.py -v
```

Before you show the TA: `git diff --stat tests/` must print nothing, and nothing in `dsa/hashmap.py` may create a list, dict or set.

Part 11 — Take-home practice (not graded)

The question bank for this week is `docs/question-bank/week13-questions.md`, with answers in `week13-answers.md`. Do the questions before opening the answers.

1. **Part G — write the code**, in `practice/week13.py`: `two_sum`, `first_repeated`, `group_anagrams`, `longest_distinct_run` and `count_subarrays_with_sum` (W13-C1 to W13-C5). Each is $O(n)$ with a hash map, and each has a large test on which an $O(n^2)$ answer crawls. Use your `ChainingHashMap` as the working storage:

```
pytest tests/test_practice_week13.py -v
```

2. **W13-S1** — draw a chaining table after every one of eight operations, including a resize.
3. **W13-S2** — the same for open addressing, with tombstones and a sweep.
4. **W13-B2** — a `_resize` that loses entries. Find the one line.
5. **W13-E2** — a hash function that returns 0 for every key: is the table wrong, or only slow? Argue it both ways, then decide.

The worked solutions are in `solutions/dsa/hashmap.py` and `solutions/practice/week13.py` — for after you have tried. `pytest --solutions tests/test_practice_week13.py` runs the tests on them.

Part 12 — Bridge to Lecture 14: graphs

Next week's Graph (in `dsa/graph.py`) stores, for every node, the list of nodes it is connected to — its **neighbours** — and the mapping from node to neighbours is **your** ChainingHashMap. Here is the idea in a script of your own (the neighbour lists are ordinary Python lists here; in `dsa/graph.py` they are DynamicArrays):

```
from dsa.hashmap import ChainingHashMap

roads = [("Mansoura", "Talkha"), ("Mansoura", "Sherbin"), ("Talkha", "Tanta"),
         ("Sherbin", "Damietta")]
neighbours = ChainingHashMap()
for a, b in roads:
    for x, y in ((a, b), (b, a)):          # a road goes both ways
        if x not in neighbours:
            neighbours.put(x, [])
            neighbours.get(x).append(y)

print(neighbours.get("Mansoura"))          # ['Talkha', 'Sherbin']
print(len(neighbours))                    # 5 towns
```

Two questions to bring to Lecture 14:

- **Which towns can you reach from Mansoura in at most two roads?** Answer it by hand from the map above. Then think about how a program would answer it for a map of a thousand towns — and why it needs to remember which towns it has **already visited**. (Hint: in what structure would you keep “visited”, so that asking “have I been here?” costs $O(1)$?)
- **Print every town in neighbours.** Run the loop twice, in two separate runs of Python. Is the order the same? Why not — and why might a graph want its own record of the order in which nodes were added?

Summary

Idea	The one line to keep
Hashing	<code>index = hash(key) % capacity</code> ; equal keys must hash equal.
Reproducibility	<code>int</code> hashes are fixed; <code>str</code> hashes change every run — <code>PYTHONHASHSEED</code> pins them.
Chaining	A linked list of Entry nodes per bucket; push on the front after checking for the key.
Chaining delete	<code>prev</code> and <code>entry</code> ; the head of a chain is the bucket slot itself.

Idea	The one line to keep
Resize	A new Array, and every key rehashed ; save next before relinking.
Linear probing	$(h + \text{step}) \% \text{capacity}$; stop at None.
Tombstones	Delete leaves TOMBSTONE; get skips it; put may reuse it, after probing on.
Load factor	Count tombstones as used; double when the live keys pass <code>max_load</code> , else sweep.
Measuring	Hits and misses stay a few probes below about 0.7, then climb steeply.
Worst case	Keys that share their low bits all collide: $O(n)$ per operation.

Answers to the checkpoints

Checkpoint 1. (a) **7, 5, 0, 0**. `hash(7)` is 7; `-3 % 8` is 5 in Python (it is -3 in C and Java); 24 is a multiple of 8; and `hash(2**61 - 1)` is 0, because Python reduces integer hashes modulo the prime $2^{61} - 1$. (b) `len(m)` is **2**, `m.get(3)` is **'b'** and `m.get(True)` is **'one'**. `3 == 3.0` and `True == 1` in Python, and equal keys have equal hashes, so `put(3.0, "b")` finds the key 3 and overwrites it, and `put(1, "one")` finds the key True and overwrites that. The table keeps the key object it stored first: `list(m)` is `[True, 3]`. A dict behaves the same way. (c) Because `hash(n)` is `n` in every run and on every machine, so anyone can check the index by hand — and your table will look exactly like the drawing. A `str` key lands in a different bucket every run.

Checkpoint 2. (a) `3 → 3`, `11 → 3`, `6 → 6`, `19 → 3`, `14 → 6`. Bucket 3 holds **19 → 11 → 3** (the newest at the front) and bucket 6 holds **14 → 6**; the other six buckets are None. (b) $5/8 = \mathbf{0.625}$: below 0.75, so no resize. (c) $27 \% 8 = 3$. It compares **3 entries** — 19, 11, 3 — reaches the end of the chain, and raises `KeyError: 27`.

Checkpoint 3. After `delete(11)`: bucket 3 is **19 → 3** — 11 was in the middle, so `prev.next` (19's next) now skips to 3. After `delete(19)`: bucket 3 is **3** — 19 was the head, so the bucket slot itself now points at 3; that is the head case. `delete(27)` walks bucket 3 to its end and raises `KeyError: 27`. Size: 5, 4, 3.

Checkpoint 4. (a) `put 1`: bucket 1 = 1. `put 5`: bucket 1 = `5 → 1`. `put 2`: bucket 2 = 2 (load 0.75, no resize — the test is `>`). `put 6`: $4/4 = 1.0 > 0.75$, so the table resizes, to **8**. (b) `1 → 1`, `5 → 5`, `2 → 2`, `6 → 6`: four chains of one, in buckets 1, 2, 5 and 6. (The collision between 1 and 5 has gone: they differ by 4, not by 8.) (c) Copied across, 5 would still sit in bucket 1 (behind nothing, in front of 1), but `get(5)` computes $5 \% 8 = 5$, finds bucket 5 empty and raises `KeyError: 5`. `len(m)` is right because `_size` was never touched: the keys are all still in the table — just not where anyone will look for them.

Checkpoint 5. (a) `3 → slot 3` (1 probe). `11 → home` 3 is taken, `slot 4` (2 probes). `19 → 3`

and 4 taken, slot 5 (3 probes). $4 \rightarrow$ home 4 is taken by 11, 5 by 19, slot 6 (3 probes). Slots: `_ _ 3 11 19 4 _`. (b) $12 \% 8 = 4$: probes **4** (11), **5** (19), **6** (4), **7** (None) — 4 probes, stops at the never-used slot 7, and `get(12)` raises `KeyError` (`get(12, None)` returns None). (c) A key is not necessarily in its home slot, so `get(4)` cannot look in slot 4 and give up: it must follow the same probe sequence as `put` did, until it finds 4 or reaches None. Keys from **different** homes (3 and 4) have merged into one run of four: primary clustering.

Checkpoint 6. (a) Slots: `_ _ _ 3 † 19 4 _`. `get(19)` probes **3** (3, not it), **4** (tombstone: skip), **5** (19: found) — 3 probes. (b) The load check: $(3 \text{ live} + 1 \text{ tombstone} + 1) / 8 = 0.625$, not above 0.66 — no resize. Probes: **4** (tombstone: remember it, go on), **5** (19), **6** (4), **7** (None: stop). 12 goes into the remembered tombstone, slot **4**: `_ _ _ 3 12 19 4 _`, and the tombstone count is back to 0. (c) With None in slot 4, `get(19)` probes 3, then 4: None — it stops and raises `KeyError`: 19. `get(4)` is even worse off: its home slot is 4, so it stops at the very first probe. Both keys are still in the table, in slots 5 and 6; the path to them has been cut.

Checkpoint 7. One test fails: `test_open_addressing_delete_preserves_probe_chain`, with `KeyError`: 8. It is the only test that deletes a key that **another key probed past**: 0 and 8 both hash to 0, 8 sits in slot 1, and emptying slot 0 hides it. `test_delete[OpenAddressingHashMa]` deletes the only key in the table, so there is no probe chain to cut; `test_delete_missing_key_raises` deletes nothing. The other 16 pass.

Checkpoint 8. (a) **3 tombstones**, in slots 5, 6 and 7. The fifth `put`'s check counted $(0 \text{ live} + 4 \text{ tombstones} + 1) / 8 = 0.625$ — fine; after that delete there are 5 tombstones. The sixth `put`, of 5, counted $(0 + 5 + 1) / 8 = 0.75 > 0.66$ while the live keys alone, $1/8$, were fine: so the table was **rebuilt at capacity 8**, sweeping all five tombstones, and 5 went into its home slot. Then 5, 6 and 7 were each deleted: tombstones in 5, 6, 7. The capacity never grew. (b) Every one of the 8 slots holds a tombstone, and there is no None left. `get(100)` probes **all 8 slots** before its `for` loop over `_probe` runs out: every miss is now $O(\text{capacity})$. A `while self._keys[i] is not None`: loop never finds a None and **never ends** — the test run hangs.